Secure Remote Authentication via SSH with Asymmetric Keys

Establishing trusted cryptographic baselines for server administration

Mihuț Luca-Adrian

What is SSH?

Understanding remote management and client-server architecture

Remote Management

Securely access and administer Linux servers via the command-line interface without physical hardware access, operating encrypted on TCP port 22.

Initiating a Connection

Establish immediate connection using clean terminal syntax:
`ssh user@ip_address`

Password Vulnerabilities

Traditional password authentication is weak, leaving server entry points highly susceptible to automated Brute-Force and Dictionary attacks.


```
luca@MacBook-Air---Luca [15:48:45] [~]
[→ % ssh luca@192.168.1.200
```

```
# luca @ debian-eos in ~ [15:48:50]
$
```

Asymmetric Key Cryptography Mechanism

How SSH secures remote authentication using complementary key pairs

The Private Key KEEP SECRET

Remains exclusively on your local client machine (e.g., MacBook). This file must be kept strictly confidential and never shared or transmitted.

The Public Key SHARE FREELY

Copied directly to target remote servers and appended to the local authorized storage location:
`~/.ssh/authorized_keys`

1. The Server Challenge

When attempting to log in, the server generates a random message (challenge) and encrypts it using your stored **Public Key**.

2. The Client Response

Your local client decrypts the challenge using the matching **Private Key**. The server verifies this without the private key ever crossing the network.

Key Generation & Configuration

Automating secure remote identity establishment using standard SSH tooling

STEP 1

Generate Client Keys

The system generates a cryptographically strong asymmetric key pair on your local client machine.

```
ssh-keygen -t ed25519
```

STEP 2

Transfer Public Key

Securely copy and install the generated public key onto the target remote server's credentials store.

```
ssh-copy-id
luca@192.168.1.200
```

STEP 3

Automatic Storage

The client's key is cleanly appended to the remote server file:

```
~/.ssh/authorized_keys
```

Resulting:

Easier Authentication: Instant passwordless remote shell connection without entering credentials.

Hardening & Securing the SSH Server

Modifying /etc/ssh/sshd_config to establish a robust cryptographic baseline

Disabling Password Logins

Enforces strict key-only remote authentication by completely dropping legacy login interfaces.

```
PasswordAuthentication no
```

Restrict Direct Root Login

Mitigates brute-force entry attempts by disallowing remote attackers from targeting the superuser account directly.

```
PermitRootLogin no
```

Apply Changes: Restart the SSH daemon to enforce configuration updates immediately: `sudo systemctl restart sshd`

Workflow Integrations

Streamlining server administration and development environments using VS Code Remote-SSH

Client Configuration File ~/.ssh/config

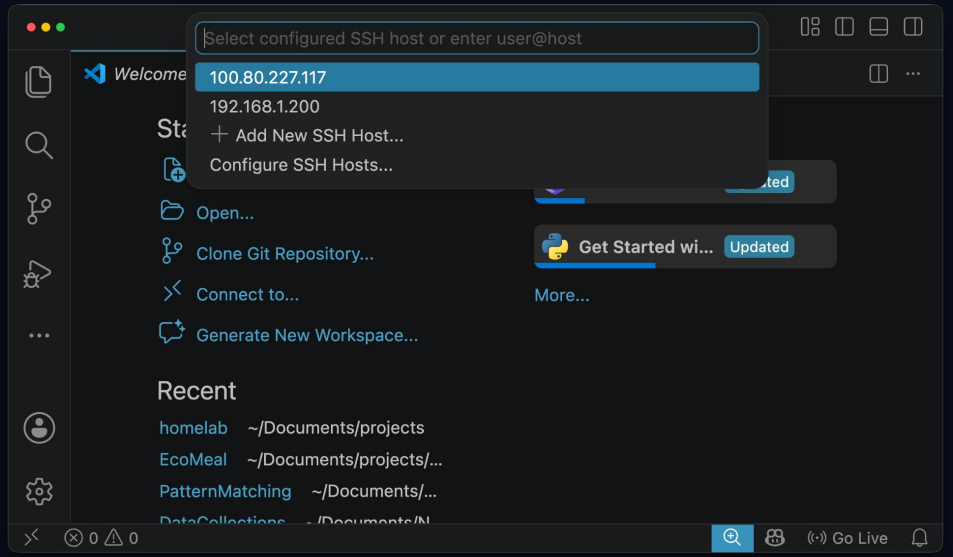
Creating custom aliases to bypass typing long IP addresses and ports, enabling instant, structured connections.

```
Host debian-eos
  HostName 192.168.1.200
  User luca
```

Remote Development & Admin

Transforming server administration with interactive desktop integrated environment capabilities.

- Edit remote server files seamlessly with GUI
- Access Debian server files and folders interactively
- Run native terminal sessions securely in-editor



DEMO

Thank you!

Bibliography:

https://www.openssh.com/
https://code.visualstudio.com/docs/remote/ssh
https://www.youtube.com/watch?v=bfwfRCCFTVI
https://www.youtube.com/watch?v=dPAw4opzN9g
https://www.youtube.com/watch?v=U_uiVyF6MEs